COMP 018: Database Administration - Activity 5 Progress Report

Database Maintenance, Performance Monitoring, and Query Analysis

Project Identification & Metadata

Course	COMP 018 - Database Administration
Evaluating Group	Group 4: Olivarez, Abangan, Pocsidio, Infantado, and Bartolata
Evaluating Group Repository	https://github.com/Nycri/brewpos
Target Project Monitored	Restaurant Online Ordering & Table Reservation System (Mesa / Binary Sisters)
Target Project Group	Group 10: Sandoval, Cellyn; Callos, Kathlen Mae; Romingquit, Arika; Tambio, Jenny Mae; Tungol, Jaimie; Gordonas, John Benedick
Target Project Repository	https://github.com/kathlenmaecallos11-arch/Restaurant-Online-Ordering-Table-Reservation-System
Primary Database	grp_binarysisters_projectdb (PostgreSQL 18.6 on x86_64 Windows, compiled by MSVC)
Restored Test Database	grp_binarysisters_projectdb_restore_test

ACTIVITY 5: Database Maintenance and Monitoring

2.1 Database Health & Overview Inspection

Database health inspection was conducted on PostgreSQL 18.6 running on a 64-bit Windows environment. Analysis confirmed that the database occupies a total disk footprint of approximately 9.3 megabytes across eight base tables, four views, and eight sequences within the public schema. Table storage analysis showed that customer_order and order_item represent the largest relations at 48 kilobytes each, followed by reservation at 32 kilobytes, and menu_item, category, customer, staff, and role at 24 kilobytes each.

2.2 Connection, Session & Activity Monitoring

Database statistics recorded one active connection with 74 committed transactions and 7 rolled back transactions. Zero lock conflicts and zero deadlocks were recorded across sessions, confirming stable transaction processing. The rolled back transaction count reflected expected error conditions intentionally triggered during negative security verification testing in Activity 4. System catalog queries verified that active

roles and memberships accurately reflect access control rules.

2.3 Table Statistics & Bloat Inspection

Pre-maintenance table statistics revealed dead tuple accumulation within the customer_order and reservation tables resulting from order status changes, payment processing, and table reservation lifecycle updates executed during Activity 4 verification testing. Obsolete row versions across active relations confirmed the necessity of performing routine vacuuming and statistics updating operations.

2.4 Log Review & Issue Identification

Review of database logs verified that unauthorized access attempts during Activity 4 security testing were recorded by the engine. Logged events included permission denials on update, insertion, and deletion commands attempted under reporting credentials, blocked database creation requests, and denied cross-database connection attempts. Server background processes completed cleanly without system crashes or storage warnings.

2.5 Routine Maintenance Operations

Routine maintenance was conducted by executing system statistics updating and tuple vacuuming operations across target tables. Initial testing confirmed that non-owner application runtime accounts are denied execution of maintenance commands, proving that maintenance tasks require administrative credentials.

Full database statistics updates executed cleanly under administrative credentials. Standard vacuuming on customer_order, order_item, and reservation scanned the relations, removing obsolete row versions and updating frozen transaction counts. Vacuum processing on menu_item and category removed dead tuples and truncated empty pages, restoring storage efficiency.

Full table rewriting vacuum operations were omitted because such commands acquire exclusive locks that block concurrent application read and write operations. Because relations reside within single disk pages, standard non-blocking vacuuming reclaimed dead tuples effectively without disrupting application availability.

2.6 Query Performance Analysis

Performance analysis was conducted on a public dining query filtering active menu items by category and sorting results by popularity. The query executed in 0.071 milliseconds with complete buffer cache hit efficiency (4 buffer hits, 100% cache hit ratio). The query planner selected sequential scans over index lookups because underlying data fits inside a single memory page, avoiding index page processing overhead.

Analysis of a multi-table dining order breakdown query querying v_order_details with full customer, item, and category joins yielded an execution time of 0.618 milliseconds with zero physical disk reads. The query engine executed chained hash joins within dedicated memory structures without disk spills.

2.7 Prioritized Recommendations & Technical Justification

The current index configuration is appropriate for current operational requirements, as active queries execute under 0.62 milliseconds with complete buffer cache hits. Immediate index additions are unnecessary and would introduce write overhead during high-volume customer checkout operations. If order histories expand into tens of thousands of records in production, adding a composite B-Tree index on customer_order(customer_id, order_date) will optimize customer history lookups. Deploying a scheduled weekly background maintenance task will maintain page efficiency during low-traffic periods.

CLASS GOOGLE SHEETS TRACKER ENTRY (Activity 5)

Column Header	Value
Project	Group 10 - Restaurant Online Ordering & Table Reservation System (Binary Sisters)
Database inspected	grp_binarysisters_projectdb (PostgreSQL 18.6)
Account and role used	postgres (DBA Maintenance), rors_app (Staff Admin), rors_report (Auditor), rors_maint (Maintenance)
Health check result	Healthy (0 conflicts, 0 deadlocks, 9.3 MB footprint, 100% buffer cache hits)
Maintenance performed	ANALYZE; VACUUM (VERBOSE, ANALYZE) on customer_order, order_item, reservation via postgres
Query analyzed	Menu item search by category (Seq scan, 0.071 ms) & Multi-table order details join (Hash joins, 0.618 ms)
Recommendation priority	No action required (Current queries execute in < 0.62 ms with 100% buffer cache hits)
Evidence folder link	https://github.com/Nycri/brewpos/tree/main/activity-5/Evidence-act5
Feedback sent to owners	(05/11/2026)
PDF progress document	Activity5_Progress_Report_Group10.pdf
Last updated	2026-11-05

AI USAGE DECLARATION

Google Gemini 3.8 Flash was utilized during project evaluation to inspect database schema definitions, evaluate relational hierarchy structures, formulate role verification test scenarios, analyze

database maintenance operations and query performance execution plans, verify backup creation and restoration procedures, and structure technical report documentation. All database statements, configuration checks, and verification procedures were directly executed and validated against PostgreSQL 18.6 within the local database environment.